Do 'Funciona' ao 'É Robusto'

Dominando a Qualidade de Código no ABAP Moderno para S/4HANA



“Funcionar na minha máquina” não é mais suficiente. Em ambientes ERP críticos, um código precisa ser mais do que funcional. Ele deve ser seguro, eficiente e pronto para o futuro. Esta é a jornada para transformar seu código, e sua carreira.

NotebookLM

Apresentando a Tríade da Qualidade ABAP

A base do desenvolvimento profissional não é uma única ferramenta, mas uma filosofia de três pilares que garantem a integridade do software em esteiras de CI/CD.

Conformidade

O código segue as regras?
É seguro e pronto para a nuvem?



Correção

O código faz o que a regra de negócio exige?

Performance

O código é eficiente sob carga?

NotebookLM

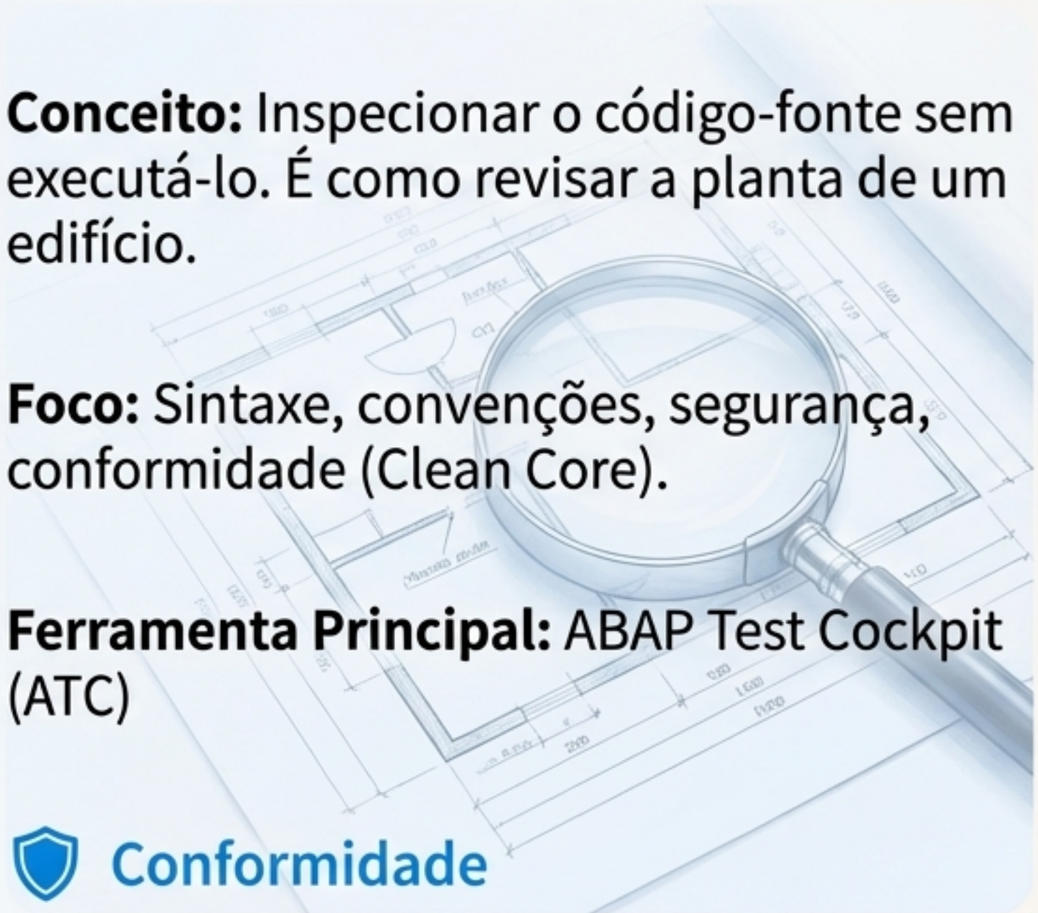
As Duas Lentes da Análise de Código

Análise Estática (O Arquiteto)

Conceito: Inspecionar o código-fonte sem executá-lo. É como revisar a planta de um edifício.

Foco: Sintaxe, convenções, segurança, conformidade (Clean Core).

Ferramenta Principal: ABAP Test Cockpit (ATC)



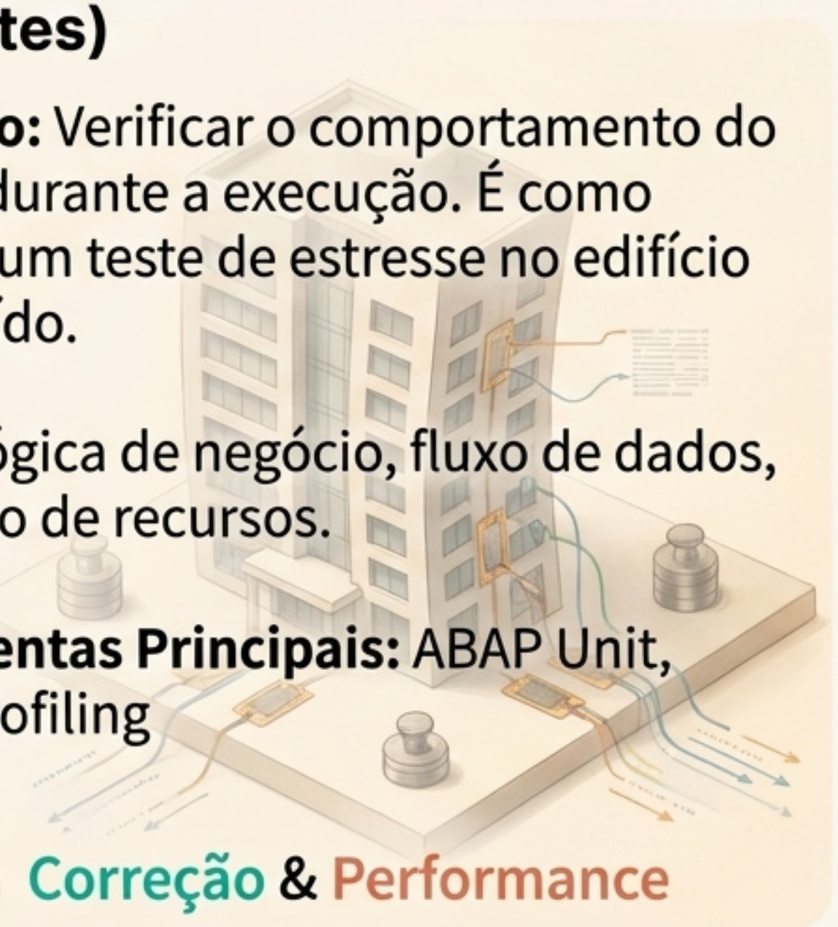
Conformidade

Análise Dinâmica (O Engenheiro de Testes)

Conceito: Verificar o comportamento do código durante a execução. É como realizar um teste de estresse no edifício construído.

Foco: Lógica de negócio, fluxo de dados, consumo de recursos.

Ferramentas Principais: ABAP Unit, ABAP Profiling



Correção & Performance

NotebookLM

Pilar 1: Conformidade com ABAP Test Cockpit (ATC)

O Guardião Aut**omatizado** da Qualidade.

O ATC é o inspetor de qualidade da SAP. Ele utiliza a **engine do Code Inspector (SCI)** para ler seu código e encontrar problemas antes que eles cheguem em produção.



O que o ATC verifica?

- **Sintaxe e Modernização:** Uso de comandos obsoletos vs. sintaxe moderna.
- **Performance (HANA):** Detecção de `SELECT` dentro de `LOOP` e outros anti-padrões.
- **Segurança:** Falta de `AUTHORITY-CHECK` e vulnerabilidades de SQL Injection.
- **Robustez:** Código morto (`Dead Code`) e tipos incompatíveis que podem gerar Dumps.
- **Cloud Readiness:** Uso de objetos não liberados e acessos proibidos no ABAP Cloud.

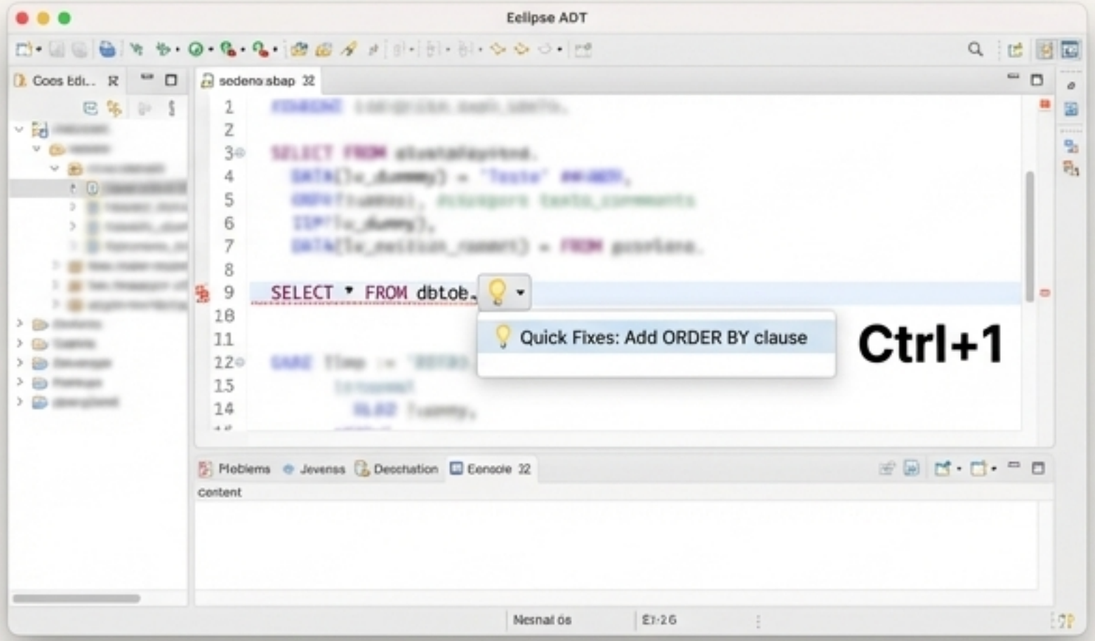
NotebookLM

Gerenciando e Corrigindo Descobertas do ATC

O ATC não apenas aponta erros, ele ajuda a corrigi-los e permite gerenciar exceções de forma inteligente.

Quick Fixes (Ctrl+1)

Acelere as correções. No ADT, muitos erros, como a falta de `ORDER BY` em um `SELECT`, podem ser corrigidos automaticamente com um atalho de teclado.



Isenções (Pragmas e Pseudo-Comentários)

Para falsos-positivos ou desvios intencionais, instrua o ATC a ignorar uma verificação específica diretamente no código.

```
" Silencia o aviso de texto hardcoded
DATA(lv_dummy) = 'Teste' ##NO_TEXT.

" Ignora uma verificação de performance específica
SELECT * FROM dbtab. "#EC CI_PERF_SELECT_STAR
```

NotebookLM

Pilar 2: Correção com ABAP Unit

A Rede de Segurança Contra Regressões.



Se o ATC inspeciona a forma, o ABAP Unit valida a função. Testes unitários garantem que seu código faz o que deveria e que futuras alterações não quebrem funcionalidades existentes. Eles respondem à pergunta: "Minha mudança quebrou algo?"

Anatomia de uma Classe de Teste

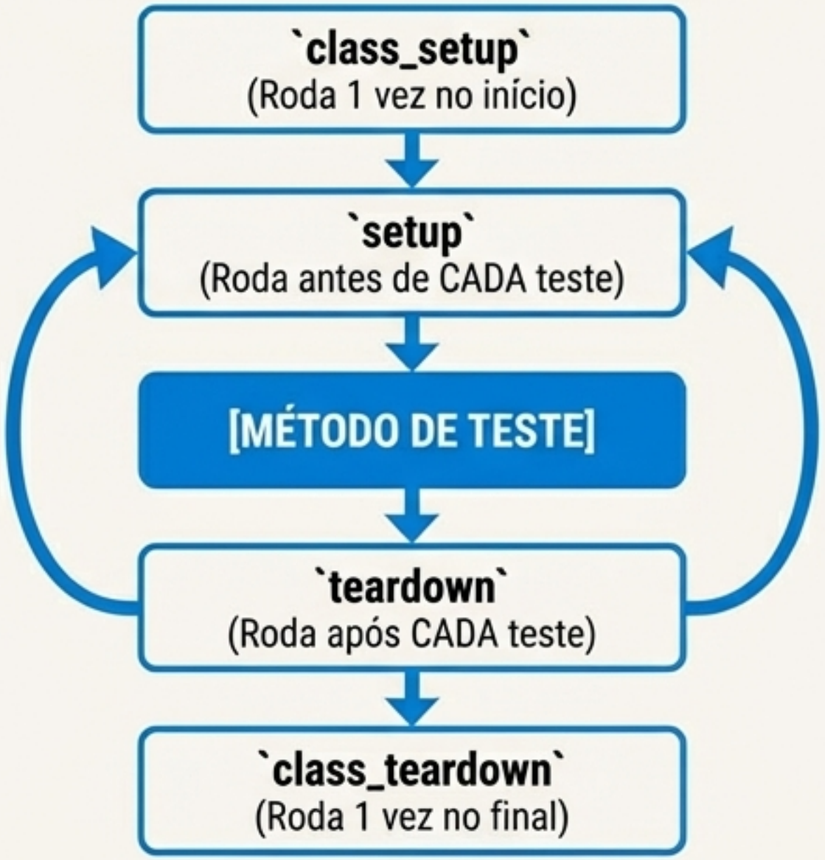
- **Definição**: Classes locais marcadas com `FOR TESTING`. Não são transportadas para produção, mas garantem a qualidade antes.
- **Configurações Essenciais**:
  - `RISK LEVEL HARMLESS`: Garante que o teste é seguro e não fará `COMMIT` no banco de dados. Obrigatório para automação.
  - `DURATION SHORT`: Assegura que o teste execute em milissegundos, mantendo o feedback rápido.

NotebookLM

O Padrão e o Ciclo de Vida de um Teste Unitário

O Ciclo de Vida (Fixtures) - Garantindo o Isolamento

Para que cada teste seja independente, o framework oferece métodos especiais para preparar e limpar o ambiente.



O Padrão AAA - A Estrutura de Todo Teste

Uma estrutura simples e poderosa para organizar a lógica de qualquer teste unitário.

A — **Arrange (Preparar)**
Crie os objetos e prepare as condições de entrada.

A — **Act (Agir)**
Execute o método que você está testando.

A — **Assert (Verificar)**
Compare o resultado obtido (`Actual`) com o resultado esperado (`Expected`).

NotebookLM

Pilar 3: Performance com ABAP Profiling

O Diagnóstico de Precisão Contra a Lentidão.

‘Olhômetro’



ABAP Profiler



Quando o código está correto e em conformidade, mas o sistema está lento, o 'olhômetro' não é confiável. O **ABAP Profiler** é a **ferramenta científica** para **medir o consumo de recursos** e encontrar o gargalo real, **real, substituindo a adivinhação** por dados concretos.

Como Funciona (Simplificado)

1. Inicie o Profiling no ADT.

2. Execute a Ação no sistema.

3. Pare e Analise o arquivo de rastreamento (Trace File).

NotebookLM

Interpretando um Rastreamento de Performance

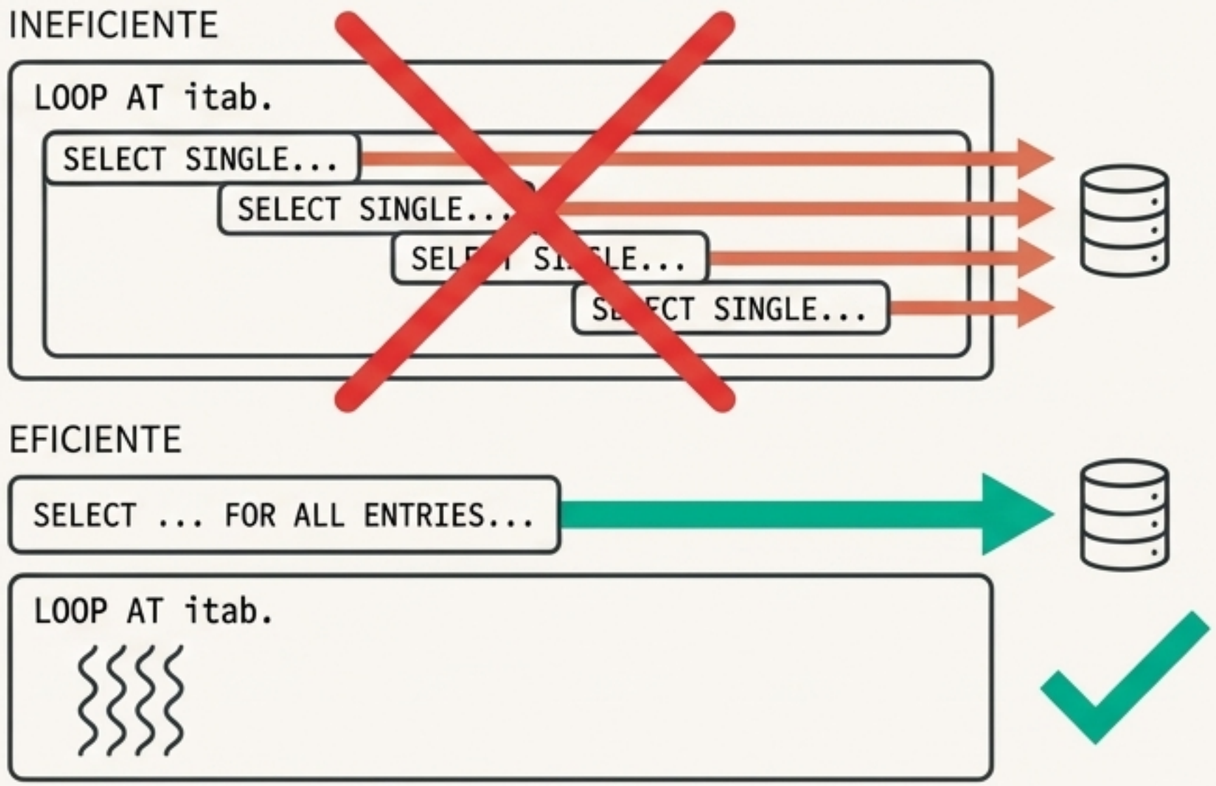
O arquivo de trace contém dados valiosos. Saiba onde focar sua atenção.

Principais Pontos de Análise

- **Hit List**
  Quais métodos foram chamados mais vezes? Um método pequeno chamado 1 milhão de vezes pode ser o problema.
- **Gross Time vs. Net Time**
  Diferencie o tempo total de um método (incluindo chamadas filhas) do tempo gasto em sua própria lógica.
- **Database Accesses**
  Veja exatamente quais `SELECT`s foram executados, quantas linhas retornaram e quanto tempo o banco demorou.

Visualizando o Problema "N+1"

A causa mais comum de lentidão: fazer um `SELECT` para cada linha de um `LOOP`.



NotebookLM

Estudo de Caso: Unindo os Pilares com `zcl_calc`

Vamos aplicar os conceitos de Correção (ABAP Unit) em uma classe simples de calculadora para ver como construir uma rede de segurança.

Código de Produção: `zcl_calc`

Uma classe pública com um método `dividir` que retorna o resultado de uma divisão e pode lançar a exceção `cx_sy_zerodivide`.

```
CLASS zcl_calc DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS dividir
        IMPORTING
           iv_a TYPE i
           iv_b TYPE i
        RETURNING
           VALUE(rv_result) TYPE p LENGTH 10 DECIMALS 2
        RAISING
           cx_sy_zerodivide.
ENDCLASS.

CLASS zcl_calc IMPLEMENTATION.
  METHOD dividir.
    " O kernel ABAP dispara cx_sy_zerodivide automaticamente
    rv_result = iv_a / iv_b.
  ENDMETHOD.
ENDCLASS.
```

NotebookLM

Construindo a Rede de Segurança para `zcl_calc`

A classe de teste local (`ltcl_test_calc`) verifica tanto o caminho feliz (sucesso) quanto o cenário de erro esperado (divisão por zero).

Fixture: Roda antes de cada teste para garantir uma instância limpa.

```
CLASS ltcl_test_calc DEFINITION FINAL FOR TESTING
  DURATION SHORT RISK LEVEL HARMLESS.
  PRIVATE SECTION.
    DATA mo_cut TYPE REF TO zcl_calc. " CUT: Class Under Test
    METHODS setup.
    METHODS test_divisao_sucesso FOR TESTING.
    METHODS test_divisao_por_zero FOR TESTING.
ENDCLASS.

CLASS ltcl_test_calc IMPLEMENTATION.
  METHOD setup.
    mo_cut = NEW #( ).
  ENDMETHOD.

  METHOD test_divisao_sucesso.
    " ARRANGE - Implícito
    " ACT
    DATA(lv_result) = mo_cut->dividir( iv_a = 10, iv_b = 2 ).
    " ASSERT
    cl_abap_unit_assert=>assert_equals(
      act = lv_result
      exp = 5
      msg = 'Divisão de 10 / 2 deve ser 5' ).
  ENDMETHOD.

  METHOD test_divisao_por_zero.
    TRY.
        mo_cut->dividir( iv_a = 10, iv_b = 0 ).
        cl_abap_unit_assert=>fail( 'Deveria ter lançado exceção' ).
    CATCH cx_sy_zerodivide.
        " SUCESSO!
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

Padrão AAA: A estrutura clara de um teste de sucesso.

Teste Negativo: Validando que o código falha da maneira correta.

NotebookLM

A Pedra de Roseta: ATC vs. ABAP Unit

Característica	🛡️ ATC (Test Cockpit)	🎯 ABAP Unit
Tipo de Análise	**Estática** (Lê o código).	**Dinâmica** (Roda o código).
O que detecta?	Erros de sintaxe, performance estrutural, segurança, convenções.	Lógica de negócio incorreta, bugs de cálculo, falhas de fluxo.
Responsabilidade	Sistema verifica automaticamente (Qualidade **passiva**).	Desenvolvedor escreve os cenários (Qualidade **ativa**).
Execução	Na liberação da tarefa ou manual.	Manual (Ctrl+Shift+F10) ou agendada.
Impacto no Transporte	**Pode impedir** a liberação da Request.	Erros não bloqueiam por padrão (mas deveriam).

NotebookLM

Guia de Campo: Glossário Técnico Essencial

CUT (Code Under Test)
O objeto específico (classe/método) que está sendo testado em um teste unitário.

Test Fixture (Setup/Teardown)
Métodos que configuram e limpam o ambiente de teste, garantindo o isolamento entre os testes.

Code Coverage (Cobertura de Código)
Métrica que indica a porcentagem do seu código produtivo que foi executada pelos testes. Cobertura alta reduz o risco de bugs.

Mocking (Test Double)
Técnica de substituir dependências reais (ex: banco de dados, APIs) por objetos simulados para testar uma unidade de código isoladamente.

Conceito Chave 1: Função do `cl_abap_unit_assert`?
Fornecer métodos estáticos (`assert_equals`, `fail`, `assert_bound`) para validar os resultados de um teste.

Conceito Chave 2: O que é `RISK LEVEL HARMLESS`?
Garante que o teste é inofensivo e não fará alterações persistentes no banco de dados (COMMIT). O framework aborta o teste se ele tentar.

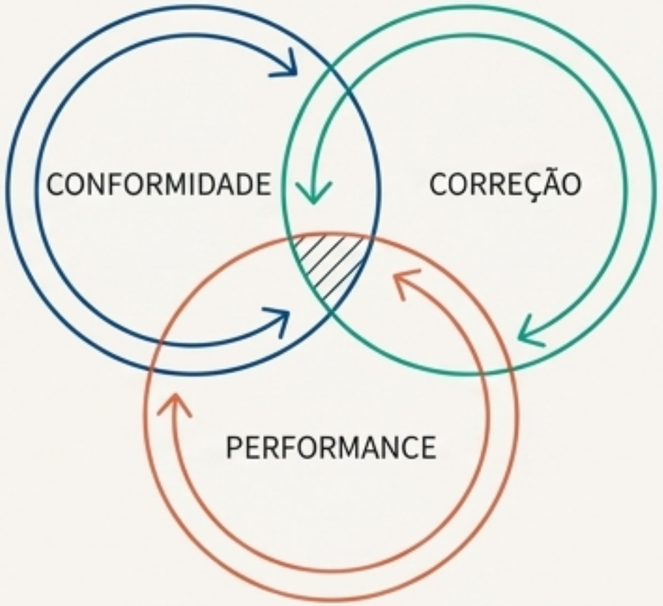
NotebookLM

Sua Tríade Para o Próximo Desenvolvimento

Antes de liberar sua próxima tarefa, faça a si mesmo as perguntas essenciais.

Conformidade

"Meu código passaria em uma verificação rigorosa do ATC? Ele está preparado para a nuvem e livre de vulnerabilidades de segurança?"



Correção

"Eu construí uma rede de segurança com ABAP Unit? Cobri o cenário de sucesso e, mais importante, os cenários de falha esperados?"

Performance

"Se este código rodar com 1 milhão de registros, ele será eficiente? Identifiquei e eliminei os gargalos como o 'N+1 SELECT'?"

Transforme a qualidade de um processo reativo para um hábito proativo.

NotebookLM